DL Sprint 4.0 – Submission Summary

Team Name: Envisage

Competition 1: Bengali Long-form Speech Recognition (ASR)

1. Overview of the Final Model

Our final Automatic Speech Recognition (ASR) system is built upon the bengaliAI/tugstugi_bengaliai-asr_whisper-medium model, which is based on the Whisper-Medium encoder-decoder transformer architecture. The model is adapted for long-form Bengali speech transcription and optimized for competition constraints.

The inference pipeline utilizes Hugging Face Transformers APIs with a chunk-based long-form transcription strategy to efficiently process extended audio segments. Each audio file is segmented into 20.1-second chunks with overlapping strides to ensure contextual continuity and improved transcription accuracy.

2. Model Configuration and Parameter Size

The Whisper-Medium architecture contains approximately 769 million parameters. This includes multi-layer transformer encoder and decoder stacks designed for multilingual speech recognition tasks.

3. Inference Strategy and Efficiency

Inference was conducted in a Kaggle GPU environment utilizing 2× NVIDIA Tesla T4 GPUs. The total runtime for complete competition inference was 56 minutes and 41 seconds.

Efficiency optimizations include:

- • Greedy decoding (num_beams=1) for faster inference.
- • Batch size of 8 for GPU utilization.
- • 16kHz resampling for model compatibility.
- • Unicode NFC normalization and zero-width character removal.
- • Fully reproducible 'Run All' notebook execution.

4. Resources and Reproducibility

Final Model Link: https://www.kaggle.com/refs/hf-model/bengaliAI/tugstugi_bengaliai-asr_whisper-medium

Dataset: DL Sprint 4.0 – Bengali Long-form Speech Recognition (Kaggle Competition Dataset)

Inference Notebook: https://www.kaggle.com/code/adilshamim8/bengali-long-form-speech-recognition/notebook

Competition 2: Bengali Speaker Diarization

1. Overview of the Final Pipeline

Our speaker diarization system, Bangla Diarizz, was constructed by modifying the pretrained Pyannote-Community-1 pipeline, changing modules, swapping models with ours, as well as adding necessary post processing.

The segmentation model was replaced with our Segmentation model, finetuned on the competition provided dataset. The finetuning was done using the diarizers library. The pyannote-embedded model was replaced with WeSpeaker ResNet34-LM. The pretrained pipeline takes care of data flow, and the internal logic (sliding window, clustering, etc) in an optimized manner.

Architecture: Input (16 kHz, Mono) -> Speaker Segmentation (outputs probability tensor of speaker A, speaker B, or both) -> Feature Aggregation → Speaker Embedding Extraction -> VBx Clustering. After this, our function stitches timestamps from the same speaker if there's a gap of less than 2s and processes it into the required format

2. Parameter Size

- Segmentation 3.0 Fine-tuned Bangla: ~1.47M parameters.
- WeSpeaker ResNet34-LM Embedding Model: ~6.63M parameters.
- Combined Pyannote pipeline components: ~ 8.1M effective parameters.

3. Inference Time

Initial runs took about 1 hour 22 mins on the competition test files. By adding an audio loader preprocessing process, so that it passes the pipeline's checks easily, inference time went down to the 35 minute 56 second mark, varying slightly in terms of seconds based on hyperparameters.

4. Resources and Reproducibility

Original Segmentation Model Link
Bangla Finetuned Segmentation Model Link
Embedding Model Link
Pyannote-C1 Pipeline Link
Inference Notebook: https://www.kaggle.com/code/gomiie/bangla-diarizz/notebook
Training Notebook: Bengali Diarization Training
Dataset: DL Sprint 4.0 – Bengali Speaker Diarization (Kaggle Competition Dataset)

The inference notebook can be run making changes (if required), then saving and committing, which will install all the dependency packages from the dependency manager automatically.